

Question1

Happy Supplies Parts Warehouse					
Customer Name: Jeff Peterson		Date: 7/1/2024			
Customer Number: HGS4587		Time: 10:30am			
Customer Type: Consumer		Employee: D. Harrison			
Part Number	Name	Type	Cage Code	Quantity Ordered	Unit Price
10654	Float Control	Plumbing	G413	4	12
10456	Modulator	Electrical	H433	3	7
10776	Hose Assembly	Plumbing	G413	7	9
10657	Float Assembly	Plumbing	G413	5	10

- assumptions:
- Customer Number uniquely identifies a customer
 - PartNumber uniquely identifies a part
 - An order is uniquely identified by Customer Number, Date, Time
 - A order can contain many parts
 - Cage Code is the storage location of the part
 - Employees can serve any customer

step 1: look at the raw table
attributes: CustomerName, CustomerNumber, CustomerType, Date, Time, Employee, PartNumber, Name, Type, CageCode, QuantityOrdered, UnitPrice

All values are atomic, there are no repeating groups, the table is in 1NF
Primary Key: CustomerNumber, Date, Time, PartNumber

OrderForm (1NF)
PK customerNumber
customerName
customerType
PK Date
PK Time
Employee
PK partNumber
Name
Type
cageCode
quantityOrdered
unitPrice

Step 2: check functional dependencies

CustomerNumber→CustomerName→CustomerType
PartNumber→Name, Type, CageCode
CustomerNumber, Date, Time→Employee
CustomerNumber, Date, Time, PartNumber→QuantityOrdered, Unit Price

Then we check the 2NF
Rule: No partial dependencies on part of a composite key, because the PK is composite, so we need to check this requirement

- Check:
- CustomerName and CustomerType depend only on CustomerNumber
 - PartName, Type, CageCode depend only on PartNumber
 - Employee depends only on CustomerNumber, Date, Time
- So all of them shows that violate 2NF, so we decompose it

2NF Decomposition
Customer 2NF
PK customerNumber
customerName
customerType

Part 2NF
PK partNumber
partName
Type
cageCode

Order 2NF
PK customerNumber
PK Date
PK Time
Employee

purchaseLine 2NF
PK customerNumber
PK Date
PK Time
PK partNumber
quantityOrdered
unitPrice

Step 3: check 3NF

3NF decomposition

Customer 3NF
PK customerNumber
customerName
customerType

Part 3NF
PK partNumber
partName
Type
cageCode

Order 3NF
PK FK customerNumber
PK Date
PK Time
Employee

purchaseLine 3NF
PK FK customerNumber
PK FK Date
PK FK Time
PK FK partNumber
quantityOrdered
unitPrice

3NF rule: no transitive dependencies
check:
customer: customerNumber→customerName; customerNumber→customerType, no customerName→other or customerType→other, so no transitive dependency
part: partNumber→partName, partNumber→ Type, partNumber→cageCode, no type→other or partName→other, so no transitive dependency
order: customerNumber, Date, Time→Employee, no employee→other, so no transitive dependency
purchaseLine: customerNumber, Date, Time, partNumber→quantityOrdered, customerNumber, Date, Time, partNumber→unitPrice, no quantityOrdered→unitPrice or unitPrice→quantityOrdered, so still no transitive dependency
Then in 3NF

Question 2

staffNo	therapistName	patNo	patName	appointment date	time	branchNo
S1011	Fred Smith	P100	Lily White	9/12/2022	10:00	M15
S1011	Fred Smith	P105	Jill Baker	9/12/2022	12:00	M15
S1024	Heidi Pierce	P108	Andy McKee	9/12/2022	10:00	Q10
S1024	Heidi Pierce	P108	Andy McKee	9/14/2022	14:00	Q10
S1032	Richard Levin	P105	Jill Baker	9/14/2022	16:30	M15
S1032	Richard Levin	P110	Jimmy Winter	9/15/2022	18:00	B13

- assumptions:
- staffNo uniquely identifies each therapist
 - patNo uniquely identifies each patient
 - a therapist can see many patients per day. but cannot see two patients at the same appointment date and time
 - Therapist may work at many branches across time but only work at one branch on given date
 - each appointment date and time will only have one therapist and one patient and occurs at one branch

step 3 2NF-3NF
rule: need to have no transitive dependencies, as I mentioned before, we need to check the patName
check:
Therapist: staffNo→therapistName, no dependency
TherapistDay: staffNo, appointmentDate→branchNo, no dependency
Appointment: patNo→patName, staffNo, appointmentDate, appointmentTime→patNo, so there is a transitive dependency, which violates 3NF
so we move patient data into its own relation

Therapist 3NF	therapistDay 3NF	Appointment 3NF	Patient 3NF
PK staffNo	PK FK staffNo	FK PK staffNo	PK patNo
therapistName	PK appointmentDate	PK appointmentDate	patName
	branchNo	PK appointmentTime	
		FK patNo	

now there is no non key attribute depends on another non key attribute, so it is in 3NF

step1: 1NF
each field is atomic, no repeating groups, so the table is in 1NF
since the appointment time slot for a therapist should be unique, so the record is uniquely identified by staffNo, appointmentDate, appointmentTime as PK

appointmentRecord (1NF)
PK staffNo
therapistName
patNo
patName
PK appointmentDate
PK appointmentTime
branchNo

- Step 2: identify dependencies
- staffNo→therapistName
 - patNo→patName
 - staffNo, appointmentDate, appointmentTime →patNo
 - staffNo, appointmentDate→branchNo
- since the PK is composite, so we must check each attribute and see whether the relationship is in 2NF .

Check:
therapistName depends only on staffNo, and staffNo is only part of the PK, so it has partial dependency, which violates 2NF
branchNo depends only on staffNo and appointmentDate, which missing appointmentTime, so still depends on part of the composite key, also a partial dependency and violates 2NF
patNo depends on the full key so it is ok for 2NF
patName is determined by patNo, not directly by part of the PK, but for 3NF we may need to check that

Therapist 2NF
PK staffNo
therapistName

therapistDay2NF
PK staffNo
PK appointmentDate
branchNo

Appointment2NF
PK staffNo
PK appointmentDate
PK appointmentTime
patNo
patName

now therapistName is only related to staffNo, branchNo is only related to staffNo, appointmentDate in these separate table, so we have removed the partial dependencies

Question 3

eNo	contractNo	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

- Assumptions:
- eNo uniquely identifies an employee
 - contractNo uniquely identifies a contract
 - each contract applies to only one event
 - an event may have multiple contracts
 - each event has only one event location
 - an employee may work on many contracts
 - the hours is specific to an employee working under a specific contract

step3: 2NF→3NF
rule: no transitive dependency
check:
eNo→eName, no transitive dependency
contractNo→eventNo, eventNo→eventLoc, so there is a transitive dependency, which violate 3NF
eNo, contractNo→hours, no transitive dependency

Event 3NF	Employee (3NF)	Contract (3NF)	workJob (3NF)
PK eventNo	PK eNo	PK contractNo	PK FK eNo
eventLoc	eName	FK eventNo	PK FK contractNo
			hours

step1: 1NF
all attributes are atomic, and there is no repeating groups, so the table is already in 1NF

Record (1NF)
PK eNo
PK contractNo
hours
eName
eventNo
eventLoc

- step2: 1NF→ 2NF
find dependencies:
- eNo→eName
 - contractNo→eventNo
 - eventNo→eventLoc
 - eNo, contractNo→hours
- 2NF rule is there is no partial dependency
- eNo→eName, eNo is part of the composite key, so that violate 2NF
 - contractNo→eventNo, contractNo is part of the composite key, so also violate 2NF
 - eventNo→eventLoc, eventNo depends on contractNo, only part of the composite key, so also violate 2NF
 - eNo, contractNo→hours, which depends on full PK, so that doesn't violate 2NF

Employee (2NF)
PK eNo
eName

Contract (2NF)
PK contractNo
eventNo
eventLoc

workJob (2NF)
PK eNo
PK contractNo
hours

Now, we removed all partial dependencies, so the tables are in 2NF